مكتب التكوين المهني و إنعاش الشغل
Office de la formation professionnelle et de la promotion du travail

Direction Régionale : Casablanca – Settat *Cité des Métiers et des Compétences Nouaceur*

Contrôle Continu N°1 : Moteurs NLP
Année de formation : 2026/2027

Filière : Intelligence Artificielle – Développeur Chatbot
Niveau : Technicien Spécialisé
Groupe : DIA_DC_TS_201
Nom et Prénom : .....

Durée : 2H00
Variante : A
Barème : ..... / 20
Documents : Non autorisés

Partie I : Questions à Choix Multiples (QCM) (8 points)

*Pour chacune des 8 questions suivantes, cochez **la seule bonne réponse**. Chaque bonne réponse rapporte **1 point**.*

1. **(1 pt)** Quelle était la principale limite des premiers systèmes NLP (années 1950–1980), fondés sur des règles grammaticales écrites à la main ?
   ☐ Ils exigeaient des millions de phrases étiquetées pour ajuster leurs règles.
   ☐ Ils étaient trop complexes à maintenir et incapables de généraliser les imprévues.
   ☐ Ils transformaient tous les mots en vecteurs numériques continus (Embedding).
   ☐ Ils traitaient uniquement des signaux vocaux et non du texte écrit.

2. **(1 pt)** En linguistique appliquée au NLP, quelle affirmation distingue correctement un **phonème** d'un **morphème** ?
   ☐ Le phonème est un mot entier du dictionnaire, alors que le morphème est un signe de ponctuation.
   ☐ Le phonème porte le sens grammatical (préfixe, pluriel), alors que le morphème désigne un son oral.
   ☐ Le morphème est la plus petite unité sonore distinctive, et le phonème est la plus petite unité porteuse de sens.
   ☐ Le phonème est la plus petite unité sonore distinctive, et le morphème est la plus petite unité porteuse de sens.

3. **(1 pt)** Un client écrit : *« Ma fibre est coupée »*. Le module **X** extrait `{intent: "panne", offre: "fibre"}`. Le module **Y** produit ensuite la réponse : *« Un technicien arrive sous 4h. »* Identifiez **X** et **Y** :
   ☐ X = NLG et Y = NLU
   ☐ X = Stemming et Y = Parsing
   ☐ X = Speech-to-Text et Y = OCR
   ☐ X = NLU et Y = NLG

4. **(1 pt)** Pour entraîner un chatbot détectant les demandes d'annulation, pourquoi faut-il éviter de supprimer aveuglément tous les **stop words** français standards ?
   ☐ Parce que cela augmente fortement la taille du vocabulaire et ralentit le calcul.
   ☐ Parce que cela ramène tous les verbes conjugués à l'infinitif et efface le temps grammatical.
   ☐ Parce que cela retire des mots de négation (*« ne »*, *« pas »*), rendant identiques *« Je veux annuler »* et *« Je ne veux pas annuler »*.
   ☐ Parce que la tokenisation ne peut plus fonctionner si les mots vides sont absents.

5. **(1 pt)** Soit la phrase *« Nous résilierons nos abonnements »*, quel résultat donne une **lemmatisation** (`token.lemma_` avec spaCy) sur le verbe et le nom ?
   ☐ `['résili', 'abonn']`
   ☐ `['résilierons', 'abonnements']`
   ☐ `['résilier', 'abonner']`
   ☐ `['verbe_futur', 'nom_pluriel']`

6. **(1 pt)** Quel pipeline décrit fidèlement l'architecture **RAG** (*Retrieval-Augmented Generation*) d'un chatbot d'entreprise ?
   ☐ Vectoriser la question → chercher les passages proches en base vectorielle → les injecter dans le prompt du LLM pour générer une réponse sourcée.
   ☐ Ré-entraîner les poids du LLM à chaque question client puis supprimer les PDF dans la base de données vetoriel pour générer une réponse sourcée.
   ☐ Scraper Wikipédia → appliquer un Stemming → renvoyer le premier mot.
   ☐ Injecter 10 000 PDF entiers sans découpage (*chunking*) dans chaque requête.

7. **(1 pt)** Pourquoi remplace-t-on souvent un réseau **SimpleRNN** par un réseau **LSTM** ou **GRU** sur des messages clients longs ?
   ☐ Parce que `SimpleRNN` ignore totalement l'ordre des mots dans une séquence.
   ☐ Parce que `SimpleRNN` subit la vanishing gradient et oublie le début de la phrase.
   ☐ Parce que `SimpleRNN` ne peut réaliser que des régressions et non de la classification.
   ☐ Parce que les couches LSTM et GRU acceptent du texte brut sans vectorisation préalable.

8. **(1 pt)** Quelle caractéristique distingue principalement une cellule **GRU** d'une cellule **LSTM** ?
   ☐ La cellule GRU comporte quatre portes internes de plus que le LSTM et consomme davantage de mémoire.
   ☐ La cellule LSTM fusionne la mémoire dans un seul état caché à l'aide de deux portes, offrant moins de paramètres et un entraînement plus rapide que le GRU.
   ☐ La cellule GRU intègre directement le `Tokenizer` et supprime le besoin d'une couche `Embedding`.
   ☐ La cellule GRU fusionne la mémoire dans un seul état caché à l'aide de deux portes, offrant moins de paramètres et un entraînement plus rapide que le LSTM.

Partie II : Web Scraping & Bibliothèque Regex (6 points)

*Associez chaque besoin technique (**Colonne A : 1 à 6**) au code Python correspondant (**Colonne B : A à F**). Écrivez la lettre choisie dans la case réponse (**1 point par association**).*

Rép.	Colonne A – Besoin technique	Code	Colonne B – Instruction Python
1	**(1 pt)** Configurer Selenium pour lancer le navigateur Chrome en arrière-plan sans fenêtre visible.	**A**	`propre = re.sub(r",(?=\d)", "", texte)`
2	**(1 pt)** Parser le code HTML d'une page statique téléchargée afin de naviguer dans ses balises.	**B**	`driver.find_elements(By.CLASS_NAME, "avis")`
3	**(1 pt)** Extraire dans une liste toutes les URLs cibles contenues dans les liens hypertextes d'une page statique.	**C**	`options.add_argument("-headless")`
4	**(1 pt)** Récupérer avec Selenium sur une page dynamique (JavaScript) tous les éléments HTML d'une même classe CSS.	**D**	`numeros = re.findall(r"\b\d{10}\b", texte)`
5	**(1 pt)** Supprimer uniquement les virgules suivies d'un chiffre (*Lookahead*), sans supprimer le chiffre lui-même.	**E**	`soup = BeautifulSoup(response.text, "html.parser")`
6	**(1 pt)** Extraire sous forme de liste Python toutes les séquences de 10 chiffres présentes dans un message client.	**F**	`text_ref= [b["href"] for b in soup.find_all("a", href=True)]`

Récapitulatif de vos réponses : 1 → ____ 2 → ____ 3 → ____ 4 → ____ 5 → ____ 6 → ____

Partie III : Correction de Code Python (GRU) (6 points)

Le script Python ci-dessous entraîne un réseau récurrent **GRU** pour classer les messages d'un chatbot selon **3 intentions** (`0 = salutation`, `1 = reclamation`, `2 = suivi_colis`).
Ce code contient **exactement 6 erreurs**. Lisez attentivement le script numéroté puis complétez le tableau de correction à la page suivante.

Consigne (6 × 1 pt = 6 points) : Pour chacune des **6 erreurs** présentes dans le code de la page 3, indiquez le **numéro de la ligne**, expliquez brièvement **la nature de l'erreur** (*0,5 pt*) et écrivez **l'instruction Python corrigée** (*0,5 pt*).

N°	Ligne	Explication de l'erreur (0,5 pt)	Instruction Python corrigée (0,5 pt)
1	.....		
2	.....		
3	.....		
4	.....		
5	.....		
6	.....		

```python
import re
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GRU, Dense

# 1. Donnees d'entrainement du Chatbot (3 intentions : 0, 1, 2)
INTENTIONS = {0: "salutation", 1: "reclamation", 2: "suivi_colis"}
messages_bruts = [
    "<div>Bonjour, comment allez-vous aujourd'hui ?</div>",
    "<p>Mon internet est coupe depuis hier !</p>",
    "Ou se trouve ma commande numero 4589 ?",
    "Bonsoir, j'ai besoin d'un renseignement svp",
    "<span>Service en panne, rien ne fonctionne</span>",
    "Quand vais-je recevoir mon colis ?"
]
labels = [0, 1, 2, 0, 1, 2]

# 2. Nettoyage du texte avec Regex
def nettoyer(texte):
    texte = re.search(r'<.*?>', ' ', texte)
    texte = texte.lower()
    return re.sub(r'\s+', ' ', texte).strip()

textes = [nettoyer(m) for m in messages_bruts]

# 3. Tokenisation et Padding
tokenizer = Tokenizer(num_words=100)
tokenizer.fit(textes)
sequences = tokenizer.texts_to_sequences(textes)
X = pad_sequences(sequences, maxlen=10)
y = np.array(labels)

# 4. Modele GRU pour classifier les 3 intentions
model = Sequential()
model.add(input_dim(Embedding=100, output_dim=8, input_length=10))
model.add(GRU(16))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X, y, epochs=30, verbose=0)

# 5. Test sur un nouveau message client
seq_test = tokenizer.texts_to_sequences([nettoyer("Mon colis tarde")])
X_test = pad_sequences(seq_test, maxlen=10)
probas = model.Production(X_test)
id_intention = int(np.argmax(probas[0]))
print("Intention predite :", INTENTIONS[id_intention])
```